



Universidade de Brasília
Departamento de Ciência da Computação

Listas

Professora: Eliza Gomes

E-mail: eliza.gomes@unb.br

Conceito

- ▶ É uma estrutura de dados linear utilizada para armazenar e organizar dados em um computador;
- ▶ Uma estrutura do tipo lista é:
 - ✓ Uma sequência de elementos do mesmo tipo
 - ✓ Pode possuir elementos repetidos
 - ✓ Pode ser ordenada ou não
- ▶ Pode possuir **N** elementos ou itens;
 - ✓ A lista está vazia se **N = 0**

Tipos de listas

- ▶ As representações diferentes para uma lista variam a depender de como os elementos são inseridos ou removidos da lista, tipo de alocação usada e tipos de acesso aos elementos;
- ▶ Quanto à inserção / remoção de elementos da lista:
 - ✓ **Lista convencional:** pode ter elementos inseridos ou removidos de qualquer lugar dela;
 - ✓ **Fila:** estrutura do tipo FIFO (*First In First Out*), os elementos só podem ser inseridos no final da lista e acessados ou removidos no início da lista;
 - ✓ **Pilha:** estrutura do tipo LIFO (*Last In First Out*), os elementos só podem ser inseridos, acessados ou removidos do final da lista;

Tipos de listas

► Quanto à alocação de memória:

- ✓ **Alocação estática:** o espaço de memória é alocado no momento da compilação do programa. É necessário definir o número máximo de elementos que a lista irá possuir;
- ✓ **Alocação dinâmica:** o espaço de memória é alocado em tempo de execução. A lista cresce à medida que novos elementos são armazenados, e diminui à medida que elementos são removidos.

► Independente de como a memória foi alocada, seus elementos são acessados das seguintes formas:

- ✓ **Acesso sequencial:** os elementos são armazenados de forma consecutiva na memória;
- ✓ **Acesso encadeado:** cada elemento pode estar em uma área distinta da memória. É necessário que cada elemento da lista armazene, além da sua informação, o endereço de memória onde se encontra o próximo elemento. Para acessar um elemento, é preciso percorrer todos os seus antecessores na lista.

Operações básicas de uma lista

- ▶ Criação da lista;
- ▶ Inserção de um elemento na lista;
- ▶ Remoção de um elemento da lista;
- ▶ Busca por um elemento da lista;
- ▶ Destruição da lista;
- ▶ Além de informações com tamanho, se está cheia ou vazia.

Lista Sequencial Estática

Lista sequencial estática

- ▶ É uma lista definida utilizando alocação estática e acesso sequencial dos elementos;
- ▶ Essa lista é definida utilizando um *array*, de modo que o sucessor de um elemento ocupa a posição física seguinte do mesmo;

Tamanho da lista => $N = 5$

Índices	0	1	2	3	4	N - 1
Dados	33	43	53	63	73	

Lista sequencial estática

► Vantagens:

- ✓ acesso rápido e direto aos elementos (índice do *array*);
- ✓ tempo constante para acessar um elemento;
- ✓ facilidade em modificar as suas informações.

► Desvantagens:

- ✓ definição prévia do tamanho do *array* e, conseqüentemente, da lista;
- ✓ dificuldade para inserir e remover um elemento entre outros dois: é necessário deslocar os elementos para abrir espaço dentro do *array*.

Lista sequencial estática

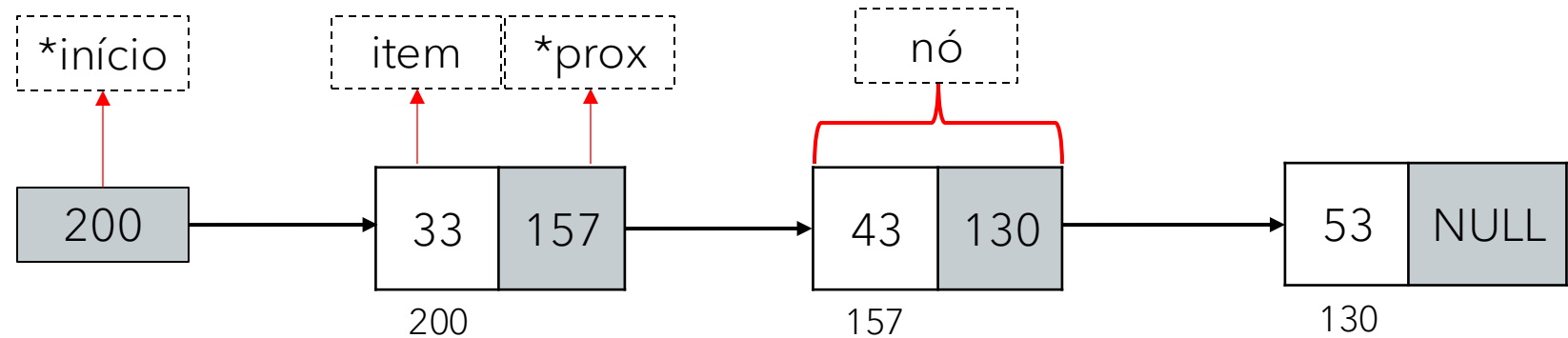
- ▶ Esse tipo de lista é utilizada nas seguintes situações:
 - ✓ listas pequenas;
 - ✓ inserção e remoção apenas no final da lista;
 - ✓ tamanho máximo da lista bem definido;
 - ✓ a busca é a operação mais frequente.

Lista Dinâmica Encadeada

Lista dinâmica encadeada

- ▶ É uma lista definida utilizando alocação dinâmica e acesso encadeado dos elementos;
- ▶ Cada elemento da lista é alocado dinamicamente, à medida que os dados são inseridos dentro da lista, e tem sua memória liberada à medida que é removido;
- ▶ Esse elemento nada mais é do que um ponteiro para uma estrutura contendo dois campos de informação:
 - ✓ um campo de **dado**: utilizado para armazenar a informação inserida na lista;
 - ✓ um campo **prox**: um ponteiro que indica o próximo elemento na lista.
- ▶ Além da estrutura que define seus elementos, essa lista utiliza um ponteiro para ponteiro para guardar o primeiro elemento da lista.

Lista dinâmica encadeada



*Endereço	Elemento	*Próximo
512	200	
157	43	130
130	53	NULL
200	33	157

Lista dinâmica encadeada

► Vantagens:

- ✓ melhor utilização dos recursos de memória;
- ✓ não é preciso definir previamente o tamanho da lista;
- ✓ não precisa movimentar os elementos nas operações de inserção e remoção.

► Desvantagens:

- ✓ acesso indireto aos elementos;
- ✓ necessidade de percorrer a lista para acessar determinado elemento.

Lista dinâmica encadeada

- ▶ Esse tipo de lista é utilizada nas seguintes situações:
 - ✓ não há necessidade de garantir um espaço mínimo para a execução da aplicação;
 - ✓ inserção e remoção em lista ordenada são as operações mais frequentes;
 - ✓ tamanho máximo da lista não é definido.

Implementação: tipo da lista

► Vamos considerar dois tipos diferentes de estruturas:

- ✓ Uma estrutura para representar a lista em si;
- ✓ Uma estrutura para representar o nó da lista.

```
typedef struct lista Lista;  
  
typedef struct listano ListaNo;  
  
struct lista {  
    ListaNo* prim;  
};  
  
struct listano {  
    int item;  
    ListaNo* prox;  
};
```

Implementação: cria lista

- ▶ A função que cria uma lista vazia deve ter como valor de retorno uma lista sem nenhum elemento;
- ▶ Como a lista armazena o ponteiro para o primeiro elemento, uma lista vazia é representada por armazenar o valor NULL , pois não existem elementos na lista;
- ▶ A função tem como valor de retorno o ponteiro para a estrutura de lista criada dinamicamente.

```
Lista* cria(void) {  
  
    Lista* l = (Lista*) malloc(sizeof(Lista));  
  
    l->prim = NULL;  
  
    return l;  
  
}
```


Implementação: insere elementos na lista

- ▶ Para cada elemento inserido na lista, devemos alocar dinamicamente a memória necessária para armazenar um novo nó e encadeá-lo na lista existente;
- ▶ A função de inserção mais simples insere o novo elemento no início da lista;
- ▶ O ponteiro que representa a lista deve ser passado como parâmetro, além da informação do novo elemento.

```
void insere(Lista* l, int v){  
  
    ListaNo* novo = (ListaNo*) malloc(sizeof(ListaNo));  
  
    novo->item = v;  
  
    novo->prox = l->prim;  
  
    l->prim = novo;  
  
}
```

Implementação: imprime elementos da lista

- Para ilustrar a implementação de uma função que percorre todos os elementos da lista, vamos considerar a criação de uma função que imprime os valores associados aos elementos armazenados numa lista.

```
void imprime(Lista* l){  
    for ( ListaNo* p=l-> prim; p!= NULL; p = p->prox)  
        printf("Item = %d\n", p->item);  
}
```

Implementação: percorre elementos da lista

- ▶ A variável auxiliar tem que ser um ponteiro, usada para armazenar o endereço de cada elemento;
- ▶ Dentro do laço da função, a variável **p** aponta para cada um dos elementos da lista, do primeiro até o último;
- ▶ Só conseguimos percorrer os elementos do primeiro ao último, pois só temos o ponteiro para o próximo nó;
- ▶ Não temos acesso randômico aos elementos da lista:
 - ✓ para acessar determinado elemento, temos que percorrer a lista desde o início, até que o elemento de interesse seja alcançado.

```
int percorre(Lista* l, int v){  
  
    for (Listano* p = l->prim; p != NULL; p = p->prox) {  
  
        if (p->item == v)  
            return 1;  
    }  
  
    return 0;        /* não encontrou o elemento */  
}
```

Implementação: remove um elemento na lista

- ▶ Se descobrirmos que o elemento a ser retirado é o primeiro da lista, devemos fazer com que o valor do primeiro elemento da lista passe a ser o ponteiro para o segundo elemento, e então podemos liberar o espaço alocado para o elemento que queremos retirar;
- ▶ Se o elemento a ser removido estiver no meio da lista, devemos fazer com que o elemento anterior a ele passe a apontar para o elemento seguinte, e então podemos liberar o elemento que queremos retirar.

Implementação: remove um elemento na lista

```
void remove(Lista* l, int v){

    ListaNo* ant = NULL; /* ponteiro para elemento anterior */

    ListaNo* p = l->prim; /* ponteiro para percorrer a lista */

    /* procura elemento na lista guardando anterior */

    while (p != NULL && p->item != v) {
        ant = p;
        p = p->prox;
    }

    if (p != NULL) {
        /* verifica se achou elemento */
        if (ant == NULL) {
            /* retira elemento do início */
            l->prim = p->prox;
        } else {
            /* retira elemento do meio da lista */
            ant->prox = p->prox;
        }
        free(p);
        /* libera elemento ( nó ) */
    }
}
```

Implementação: verifica se lista está vazia

- ▶ A função recebe a lista e retorna 1 se estiver vazia ou 0 se não estiver vazia. Como sabemos, uma lista está vazia se seu primeiro elemento é **NULL**.

```
int vazia(Lista* l){  
    return (l->prim == NULL );  
}
```

Implementação: destrói a lista

- ▶ A função primeiro libera o encadeamento dos elementos e depois libera a estrutura da lista;
- ▶ Para liberar o encadeamento, a função percorre elemento por elemento, liberando-os;
- ▶ É importante observar que devemos guardar a referência para o próximo elemento antes de liberar o elemento corrente
 - ✓ se liberássemos o elemento e depois tentássemos acessar o encadeamento, estaríamos acessando um espaço de memória que não estaria mais reservado para nosso uso.

```
void libera(Lista* l){  
  
    ListaNo* p = l->prim;  
  
    while (p != NULL) {  
  
        ListaNo* t = p->prox;           /* guarda referência para próx elemento */  
        free(p);                       /* libera a memória apontada por p */  
        p = t;                         /* faz p apontar para o próximo */  
    }  
  
    free(l);  
}
```

Implementação: TAD

- ▶ Com base na implementação, podemos criar um tipo abstrato de dados para representar uma lista encadeada de valores inteiros;
- ▶ A interface do módulo pode ser dada pelo arquivo **lista.h**:

```
typedef struct lista Lista;  
  
Lista* cria(void);  
  
void libera(Lista* l);  
  
void insere(Lista* l, int v);  
  
void remove(Lista* l, int v);  
  
int vazia(Lista* l);  
  
int percorre(Lista* l, int v);  
  
void imprime (Lista* l);
```


Implementação: main()

► Utilizando a interface `lista.h`, podemos criar um programa que utiliza as funções de lista exportadas.

```
# include <stdio.h>

# include "lista.h"

int main(void) {

    Lista* l = cria();
    insere(l, 23);
    insere(l, 45);
    insere(l, 56);
    insere(l, 78);
    imprime(l);
    remove(l, 78);
    imprime(l);
    remove(l, 45);
    imprime(l);
    libera(l);

    return 0;
}
```

```
/* cria lista vazia */
/* insere na lista o elemento 23 */
/* insere na lista o elemento 45 */
/* insere na lista o elemento 56 */
/* insere na lista o elemento 78 */
/* imprimirá : 78 56 45 23 */

/* imprimirá : 56 45 23 */

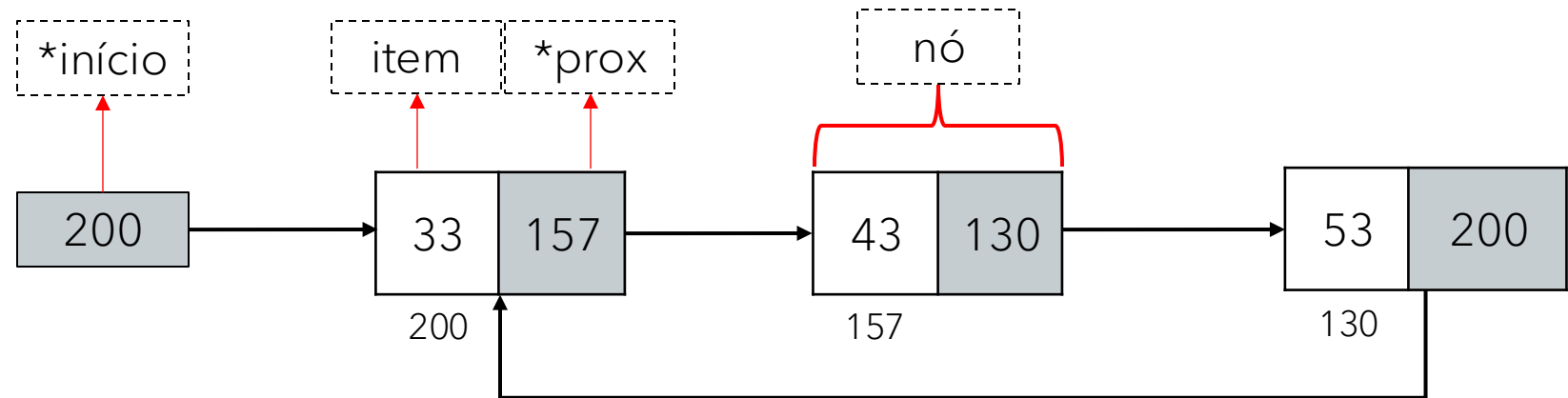
/* imprimirá : 56 23 */
```

Lista Dinâmica Encadeada Circular

Lista dinâmica encadeada circular

- ▶ Em uma lista dinâmica encadeada, após o último elemento não existe nenhum novo elemento alocado, de modo que o último elemento da lista aponta para NULL;
- ▶ Já em uma lista dinâmica encadeada circular, o último elemento tem como sucessor o primeiro elemento da lista, dando a aparência que esse tipo de lista não tem fim:
 - ✓ Nunca chegaremos a uma posição final da lista, a partir da qual não poderemos mais andar, porque depois do último elemento voltamos para o primeiro, como em um círculo.

Lista dinâmica encadeada circular



*Endereço	Elemento	*Próximo
512	200	
157	43	130
130	53	200
200	33	157

Lista dinâmica encadeada circular

► Vantagens:

- ✓ melhor utilização dos recursos de memória;
- ✓ não é preciso definir previamente o tamanho da lista;
- ✓ possibilidade de percorrer a lista diversas vezes;
- ✓ não precisa movimentar os elementos nas operações de inserção e remoção;
- ✓ não precisamos considerar casos especiais de inclusão e remoção de elementos (primeiro e último).

► Desvantagens:

- ✓ acesso indireto aos elementos;
- ✓ necessidade de percorrer a lista para acessar determinado elemento;
- ✓ a lista não possui um final definido. Logo, não há como saber se já percorremos a lista inteira.

Lista dinâmica encadeada circular

- ▶ Esse tipo de lista é utilizada nas seguintes situações:
 - ✓ não há necessidade de garantir um espaço mínimo para a execução da aplicação;
 - ✓ inserção e remoção em lista ordenada são as operações mais frequentes;
 - ✓ tamanho máximo da lista não é definido;
 - ✓ existe a necessidade de voltar ao primeiro elemento da lista depois de percorrê-la.

Implementação: verifica se lista está vazia

- ▶ Para percorrer os elementos de uma lista circular, visitamos todos os elementos a partir do ponteiro do elemento inicial até alcançar novamente esse mesmo elemento;
- ▶ Caso a lista esteja vazia ainda deve ser tratado
 - ✓ se a lista é vazia, o ponteiro para um elemento inicial vale NULL .

```
void percorre(Lista* l){  
  
    ListaNo* p = l->prim;           /* faz p apontar para nó inicial */  
  
    if (p != NULL) do {              /* se não vazia , percorre */  
        printf("%d\n", p->info);      /* imprime informação do nó */  
        p = p->prox;                 /* avança para o próximo nó */  
    } while (p != l->prim );  
  
}
```

Lista Dinâmica Duplamente Encadeada

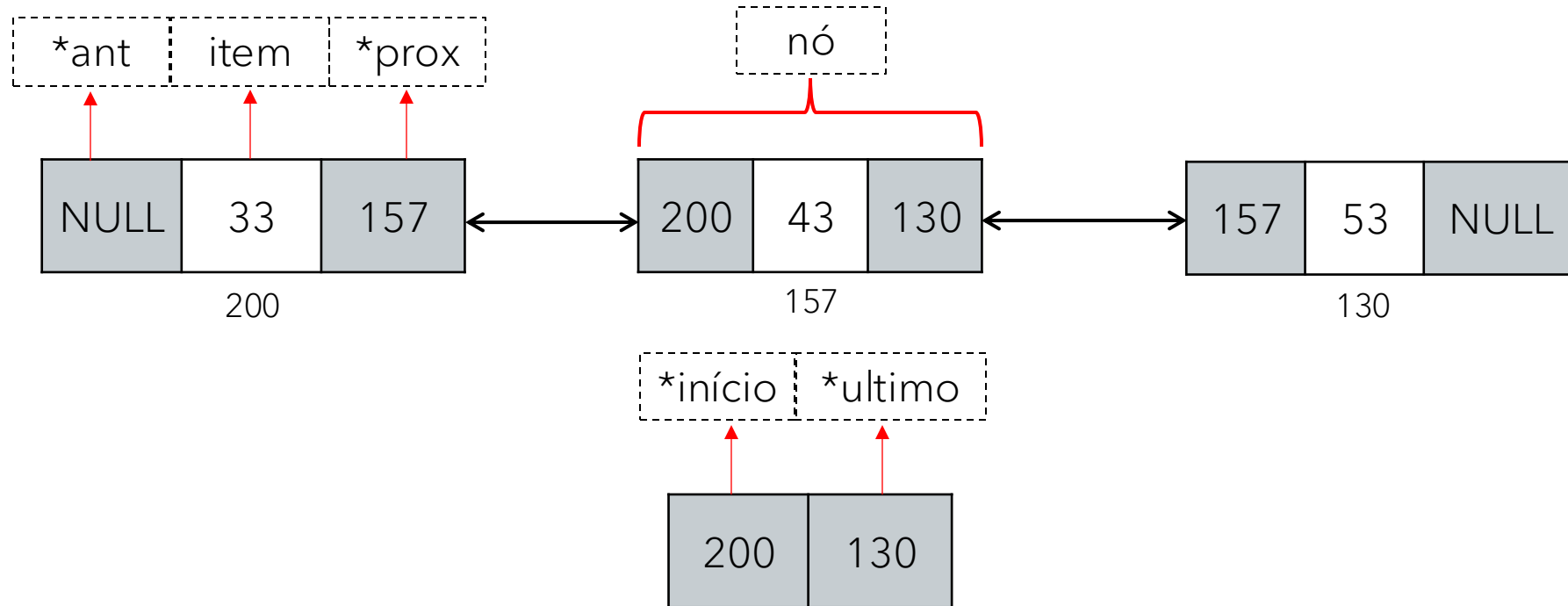
Lista dinâmica duplamente encadeada

- ▶ Uma lista dinâmica duplamente encadeada é uma lista definida utilizando alocação dinâmica e acesso encadeado dos elementos;
- ▶ Cada elemento da lista é alocado dinamicamente, à medida que os dados são inseridos dentro da lista, e tem sua memória liberada à medida que é removido;
- ▶ Essa estrutura contém agora três campos de informação:
 - ✓ Um campo de dado: utilizado para armazenar a informação inserida na lista;
 - ✓ Um campo próximo: utilizado para indicar o próximo elemento na lista;
 - ✓ Um campo anterior: idêntico ao campo próximo, mas utilizado para indicar o elemento anterior na lista.

Lista dinâmica duplamente encadeada

- ▶ A presença dos ponteiros **prox** e **ant** garantem que a lista seja encadeada em dois sentidos:
 - ✓ No seu sentido normal: aquele usado para percorrer uma lista do seu início até o seu final;
 - ✓ No sentido inverso: quando percorremos a lista de volta ao seu início.
- ▶ Por isso, ela é chamada duplamente encadeada:
 - ✓ cada elemento aponta para o seu sucessor (**prox**) e antecessor (**ant**) dentro da lista.

Lista dinâmica duplamente encadeada



*Endereço	Elemento	*Anterior	*Próximo
157	43	200	130
130	53	157	NULL
200	33	NULL	157

*Endereço	*Início	*Último
512	200	130

Implementação: tipo da lista

- ▶ Vamos considerar que queremos armazenar valores inteiros na lista;
- ▶ A estrutura da lista pode armazenar dois ponteiros:
 - ✓ Um para o primeiro nó e outro para o último nó
 - ✓ O nó da lista armazena, além da informação, ponteiros para o próximo nó e para o nó anterior:

```
typedef struct lista Lista;  
typedef struct listano ListaNo;  
  
struct lista{  
  
    ListaNo* prim;  
  
    ListaNo* ult;  
};  
  
struct listano2 {  
  
    int info;  
    ListaNo* ant;  
    ListaNo* prox;  
};
```

Implementação: cria a lista

► A função que cria uma lista vazia é simples:

```
Lista* cria_lista(void) {  
    Lista* l = (Lista*) malloc(sizeof(Lista));  
  
    l->prim = NULL;  
  
    l->ult = NULL;  
  
    return l;  
}
```

Implementação: inserir elemento da lista

- No caso da lista duplamente encadeada, as inserções no início e no final são simples e eficientes.

```
/* inserção no início */

void insere_inicio (Lista* l, int v){

    ListaNo* novo = (ListaNo*) malloc(sizeof(ListaNo));
    novo->info = v;
    novo->prox = l->prim;           /* próximo de novo é antigo primeiro */
    novo->ant = NULL;               /* anterior é nulo */

    if (l->prim != NULL){           /* testa se lista é vazia */
        l->prim->ant = novo;        /* novo é anterior do antigo primeiro */
    }else{
        l->ult = novo;             /* novo também é último */
    }
    l->prim = novo;                /* novo é primeiro */
}
```

Implementação: inserir elemento da lista

- No caso da lista duplamente encadeada, as inserções no início e no final são simples e eficientes.

```
/* inserção no final */

void insere_final ( Lista* l, int v){

    ListaNo* novo = (ListaNo*) malloc(sizeof(ListaNo));
    novo->info = v;
    novo->ant = l->ult;          /* anterior de novo é antigo último */
    novo->prox = NULL;          /* próximo é nulo , pois novo é último */

    if (l->ult != NULL){        /* testa se lista é vazia */
        l->ult->prox = novo;    /* novo é próximo do antigo último */
    }else{
        l->prim = novo;        /* novo também é primeiro */
    }
    l->ult = novo;              /* novo é último */
}
```

Implementação: inserir elemento da lista

- ▶ A função de busca recebe a informação referente ao elemento que queremos buscar e tem como valor de retorno o ponteiro do nó da lista que representa o elemento;
- ▶ Caso o elemento não seja encontrado na lista, o valor retornado é NULL.

```
/* função busca: busca um elemento na lista */  
  
ListaNo* busca (Lista* l, int v){  
    for (ListaNo *p= l->prim; p != NULL; p = p->prox){  
        if (p->info == v){  
            return p;  
        }  
    }  
  
    return NULL; /* não achou o elemento */  
}
```


Implementação: remover elemento da lista

```
/* função retira: retira elemento da lista */
void remove(Lista* l, int v) {

    ListaNo* p = busca(l,v);

    if (p == NULL){
        return;
    }

    if (p == l->prim){
        l->prim = p->prox;
    }else{
        p->ant->prox = p->prox;
    }

    if (p == l->ult){
        l->ult = p->ant;
    }else{
        p->prox->ant = p->ant;
    }

    free(p);

}
```